LAPORAN PRATIKUM

PEMOGRAMAN STRUKTUR DATA

“Pratikum Pekan 1”

Disusun Oleh:

Jose Moniz de Araujo

2511538002

Dosen Pengampu : Dr. Wahyudi, S, T, M.T.

Asisten Pratikum : Muhammad Galid Avero.

DEPARTAMENT INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGATAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas terselesaikannya laporan mingguan pecan ke-1 ini. Laporan ini disusun untuk memnuhi tugas mata kuliah **Struktur Data** yang berfokus pada pemahaman konsep dasar **Abstract Data Type(ADT).**

Pada pecan pertama ini, focus utama pembelajaran adalh memahami bagaimana Strukur data dapat didefinisikan secara logis melalui ADT sebelum dimplementasikan ke dalam kode program. Penulis menyadari bahwa laporan ini masih jauh dari sempurna, oleh karena itu kritik dan saran yang membangung sangat diharapkan.

Akhir kata, semoga laporan ini dapat bermanfaat bagi penulis khususnya, dan pembaca pada umumnya.

Padang, 01/04/2026

Jose Moniz de Araujo

DAFTAR ISI

KATA PENGANTAR 2

DAFTAR ISI 3

BAB I PENDAHULUAN 4

1.1 Latar Belakang 4

1.2 Tujuan 4

1.3 Manfaat 4

BAB II PEMBAHASAN 5

2.1 Analisis Program 5

2.2 Implemantasi Program 5

2.3 Implementasi program utama 8

BAB III PENUTUP 13

3.1Kesimpulan 13

DAFTAR PUSTAKA 14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, konsep *Abstract Data Type* (ADT) merupakan salah satu dasar penting yang harus dipahami oleh setiap mahasiswa, khususnya dalam bidang informatika. ADT memungkinkan seorang programmer untuk mendefinisikan struktur data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa harus mengetahui detail implementasinya secara langsung.

Pada praktikum yang telah dilakukan, mahasiswa diminta untuk mengimplementasikan ADT sederhana dalam bahasa pemrograman Java menggunakan aplikasi Eclipse. Salah satu studi kasus yang digunakan adalah pembuatan program "Jam", yang merepresentasikan waktu dalam bentuk jam, menit, dan detik.

Program ini tidak hanya melatih pemahaman konsep ADT, tetapi juga membantu mahasiswa dalam memahami penggunaan konstruktor, *setter*, *getter*, serta metode lain yang berkaitan dengan manipulasi data waktu. Dengan adanya praktikum ini, diharapkan mahasiswa mampu memahami bagaimana cara merancang dan mengimplementasikan suatu tipe data abstrak secara sistematis dan terstruktur.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Abstract Data Type (ADT).

2. Mampu mengimplementasikan ADT menggunakan bahasa pemrograman Java.

3. Mampu membuat program yang merepresentasikan objek "Jam".

4. Memahami penggunaan konstruktor, *setter*, *getter*, dan metode lainnya dalam Java.

1.3 Manfaat

Manfaat yang diperoleh dari pratikum ini antara lain:

1. Manambah pemahaman tentang konsep ADT dalam pemrogaraman
2. Melatih kemampuan dalam merancang program berbasis objek.
3. Meningkatkan keterampilan dalam menggunakan Bahasa pemrograman Java.
4. Menjadi dasar dalam mempelajari struktur data yang labih kompleks.

BAB II

PEMBAHASAN

2.1 Analisis Program

Program yang dibuat merupakan implementasi Abstract Data Type (ADT) dengan studi kasus objek Jam. Program ini digunakan untuk mempresentasikan waktu dalam satuan jam, menit, dan detik.

Object jam memiliki tiga atribut utama, yaitu:

1. Jam (hh) dengan rentang 0-23
2. Menit (mm) dengan rentang 0-59
3. Detik (ss) dengan rentang 0-59

Selain itu, program ini juga menyediakan berbagai operasi seperti valiadasi waktu, konversi ke detik, perbandingan waktu, operasi aritmatika waktu, serta penentuan durasi.

2.1.1. Operasi pada ADT Jam

1. Validator : mengecek apakah nilai jam valid atau tidak
2. Konstruktor : untuk membuat objek jam baru
3. Selecotor (Getter) : mengambil nilai jam, menit dan detik
4. Mutator (Setter) : mengubah nilai jam, menit dan detik
5. Konversi : mengubah waktu menjadi detik dan sebaliknya
6. Relasional : Membandingkan dua objek jam
7. Aritmatika : penjumlahan, pengurangan dan pergersan waktu
8. Durasi : mengitung selisih waktu
9. Output : menampilkan waktu dalam format string

2.2 Implemantasi Program

Berikut adalah implementasi class jam dalam bahasa java:

```
package pekan1_2511530082;

public class Jam_2511538002 {

```

1. Atribut

```
    private int hh; //0..23
    private int mm; // 0...59
    private int ss; //0...59
```

a. hh menympan jam (0-23)
b. mm menympan menit (0-59)
c. ss menympan detik (0-59)
d. mengunakan private untuk menjaga keamanan data

2. Validator

Digunakan untuk mengecek apakah input waktu valid, dan mengembalikan nilai true jka valid, false jika tidak.

```
//validator
public static boolean isValid(int h, int m, int s) {
return (0<= h && h <= 23) && (0<=m && m<=59) && (0<= s && s <= 59);
```

3. Konstruktor

Digunakan untuk membuat objek jam baru, **this** digunakan untuk membedakan variable atribut dengan parameter

```
    }
// kontruktor
    public Jam_2511538002(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;

    }
```

4. Getter (selector)

Mengambil nilai dari atribut, diguankan agar atribut tetap private tapi bisa dibaca.

```
// selektor
public int getHour() { return hh; }
public int getMinute() { return mm;}
public int getSecond() { return ss;}
```

5. Setter (mutator)

Digunakan untuk mengubah nilai atribut

```
//mutator
public void setHour(int h) { this.hh =h;}
public void setMinute(int m) {this.mm =m;}
public void setSecond(int s) { this.ss= s;}
```

6. Konversi ke detik

Mengubah waktu menjadi total detik dengan rumus : jam x 3600 + menit x detik

```
// konversi
public int toSeconds() {return hh * 3600 + mm * 60 + ss;}
```

7. Konversi dari detik ke jam

Mengubah detik menjdi format jam dan menggunakan % agar tidak lebih dari 24 jam

```
public static Jam_2511538002 fromSeconds(int total) {
    if (total < 0)  throw new IllegalArgumentException("detik negatif");
    total %= 24 * 3600; // wrap 24 jam
    int h = total / 3600; total %=3600;
    int m = total / 60; int s = total % 60;
    return new Jam_2511538002(h, m, s);

}
```

8. Relasional (perbandingan)

Membandingkan dua waktu dan menggunakan total detik sebagai acuan

Mengecek apakah dua object jam sama

```
// relasiinal
public int compareTo(Jam_2511538002 other) { return Integer.compare(this.toSeconds() , other.toSeconds()); }
public boolean equals (Object o) {
    if ( ! ( o instanceof Jam_2511538002 j)) return false;
    return hh==j.hh && mm == j.mm && ss == j.ss;

}
```

9. Aritmatika waktu

a. Mengurangi waktu, *floorMod* agar hasil tidak negative

b. Menabah 1 detik

```
// Aritmatika j
public Jam_2511538002 plus(Jam_2511538002 other) { return fromSeconds(this.toSeconds()); }
public Jam_2511538002 minus(Jam_2511538002 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.
public Jam_2511538002 nextSeconds() { return fromSeconds(this.toSeconds() + 1);}
public Jam_2511538002 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
public Jam_2511538002 prevsecond() { return fromSeconds(Math.floorMod(this.toSeconds()  - 1, 24*3600)); }
public Jam_2511538002 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n)
```

10. Durasi

Menghitung selish waktu dalam detik

```
// durasi (detik) -- bisa negatif jika this > other ? sesuai spesifikasi durasi (Jam_2511538002, jakh)
public static int durasiDetik(Jam_2511538002 jaw, Jam_2511538002 jakh) { return jakh.toSeconds() - jaw.toSecon
```

11. Output menampilakn waktu dalam format hh:mm:ss, %02d artinya selalu 2 digit

```
    //overriding ke string
    public String toString()  { return String.format("%02d:%02d:%02d", hh, mm, ss);}


    }
```

12. Outputnya :

```
<terminated> JamDriver_2511538002 [Java Application]
a                 = 23:59:50
b                 = 00:00:15
a+b               = 23:59:50
next 20s          = 00:00:10
durasi (a,b)      = -86375
a.compareTo(b) = 1
```

2.2. Implementasi program utama

Program utama digunakan untuk menjalankan dan menguji class jam yang telah dibuat

```
package-info...   perulanganWh...   GamePenjumla...   Jam_2511538...   JamDriver_2...
1  package pekan1_2511530082;
2
3  public class JamDriver_2511538002 {
4
5      public static void main(String[] args) {
6          Jam_2511538002 a= new Jam_2511538002(23, 59, 50);
7          Jam_2511538002 b= new Jam_2511538002(0, 0, 15);
8          System.out.println("a                 = " + a );
9          System.out.println("b                 = " + b);
10         System.out.println("a+b               = " + a.plus(b));
11         System.out.println("next 20s          = " + a.nextNSeconds(20));
12         System.out.println("durasi (a,b)      = " + Jam_2511538002.durasiDetik(a, b));
13         System.out.println("a.compareTo(b) = " + a.compareTo(b));
14
15     }
16
17 }
18
```

1. Pada program utama, dibuat dua objek dari class jam:

```
Jam_2511538002 a= new Jam_2511538002(23, 59, 50);
Jam_2511538002 b= new Jam_2511538002(0, 0, 15);
```

- Objek a merepresentasikan waktu 23:59:50

- Objek b mempresentasikan waktu 00:00:15

2. Menampilkan nilai jam

```
System.out.println("a                    = " + a );
System.out.println("b                    = " + b);
```

- Method toString() otomatis dipanggil
- Output dalam format hh:mm:ss

3. Operasi penjumlahann:

   Method plus() seharusnya menjumlahkan dua waktu, namun pada kode saat ini, method belum benar.

4. Penamhan 20 detik:

   Menabakan 20 detik ke waktu a, waktu awal : 23:59:50 dan ditambah 20 detik menjadi 00: 00: 10(melewati 24 jam).

5. Perbadingan waktu:

   Mambangingkan waktu a dan b dan menggunakan total detik

   Jika a > b =1, jika sama = 0 dan a<b =-1

6. Outputnya

```
<terminated> jamDriver_2511538002 [Java Application
a                    = 23:59:50
b                    = 00:00:15
a+b                  = 23:59:50
next 20s             = 00:00:10
durasi (a,b)         = -86375
a.compareTo(b) = 1
```

2.3. Implementasi program Driver interaktif

Program ini merupakan pengembangan dari drive sebelumnya, di mana pengguna dapat memasukan data jam secara lagsung melalui keyboard.

```
1 package pekan1_2511538002;
2
3 import java.util.Scanner;
4
5 public class jamDriver2_2511538002 {
```

Program menggunakan class Scanner untuk menerima input dari pengguna.

1. Penjelasan program (dtiver interaktif)

   Program ini terjadi dari beberapa bagian utama, yaitu input, proses, dan output:

a. Input data jam

```java
// 1. Input Jam Pertama
System.out.println("\n--- Input Jam 1 ---");
jam_2511538002 j1 = buatJamDariInput(input);

// 2. Input Jam Kedua
System.out.println("\n--- Input Jam 2 ---");
jam_2511538002 j2 = buatJamDariInput(input);
```

Program meminta user memasukan dua data jam dan dilakukan melalui method buatJamDariInput()

b. Menampilkan Data Jam

```java
// 3. Menampilkan Data
System.out.println("\n--- Hasil Operasi ---");
System.out.println("Jam 1 (String)      : " + j1.toString());
System.out.println("Jam 2 (String)      : " + j2.toString());
System.out.println("Jam 1 dalam detik   : " + j1.toSecond());
System.out.println("Jam 2 dalam detik   : " + j2.toSecond());
```

- Menampilakn waktu dalam format hh:mm:ss,
- Mengubah waktu menjadi total detik
- Memudahkan perhitungan dan perbandingan

c. Perbadingan dua waktu

```java
// 4. Perbandingan
int perbandingan = j1.compareTo(j2);

if (perbandingan > 0) {
    System.out.println("Status              : Jam 1 lebih lambat (setelah) Jam 2");
} else if (perbandingan < 0) {
    System.out.println("Status              : Jam 1 lebih awal (sebelum) Jam 2");
} else {
    System.out.println("Status              : Jam 1 dan Jam 2 sama persis");
}
```

Digunakan untuk membandingkan dua waktu

Output kemungkinan :

>0 = jam lebih lambat

< 0 = jam 1 lebih cepat

= 0 = sama

d. Operasi durasi

```
System.out.println("Durasi (J1 ke J2)    : " +
        jam_2511538002.durasiDetik(j1, j2) + " detik");
```

Menghitung selish waktu dalam detik dan bisa bernilai negative jika urutan waktu terbaik.

e. Operasi waktu

```
jam_2511538002 jNext = j1.nextSecond();
System.out.println("Jam 1 Detik Berikutnya : " + jNext);

jam_2511538002 jPrev = j1.prevSecond();
System.out.println("Jam 1 Detik Sebelumnya : " + jPrev);
```

1. nextSecond() = menabah 1 detik
2. prevSecond() = mengurangi 1 detik

f. Operasi penjumlahan

```
// 6. Penjumlahan
jam_2511538002 jHasilPlus = j1.plus(j2);
System.out.println("Hasil J1 + J2       : " + jHasilPlus);
```

Menjumlahkan dua waktu dan namun masih ada kesalahan pada method Plus()

g. Fungsi pembantu

```
 */
private static jam_2511538002 buatJamDariInput(Scanner sc) {
    int h, m, s;
```

Method ini digunakan untuk:

- Input jam

- Validasi
- Membuat objek Jam

Keunggulan: Kode lebih rapi dan Tidak berulang (modular)

TUGAS PRATIKUM PEKAN 1

1. Class mobil_2511538002

```java
package pekan1_2511538002;

public class mobil_2511538002 {
    // Atribut
    private String nama;
    private int tahun;
    private int cc;
    private long harga;
    private String merk;
```

a. Berfungsi untuk mengelompokkan class agar lebih terorganisir dalam satu project.

b. public → class bisa diakses dari luar package

c. mobil_2511538002 → nama class

d. private → hanya bisa diakses dalam class (encapsulation)

e. nama → nama mobil

f. tahun → tahun produksi

g. cc → kapasitas mesin

h. harga → harga mobil

i. merk → merek mobil

2. konstruktor

```java
    // Konstruktor
    public mobil_2511538002(String nama, int tahun, int cc, long harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }
```

a. Konstruktor digunakan untuk inisialisasi objek saat pertama dibuat

b. this.nama = nama; → membedakan atribut dengan parameter

3. Getter(selector)

```java
// --- SELEKTOR (Getter) ---
public String getNama() { return nama; }
public int getTahun() { return tahun; }
public int getCc() { return cc; }
public long getHarga() { return harga; }
public String getMerk() { return merk; }
```

a. Digunakan untuk mengambil nilai atribut

b. Mendukung konsep encapsulation

4. Setter (mutator)

```java
// --- MUTATOR (Setter) ---
public void setNama(String nama) { this.nama = nama; }
public void setTahun(int tahun) { this.tahun = tahun; }
public void setCc(int cc) { this.cc = cc; }
public void setHarga(long harga) { this.harga = harga; }
public void setMerk(String merk) { this.merk = merk; }
```

a. Digunakan untuk mengubah nilai atribut

5. Method tamilkaninfo()

```java
// Method untuk menampilkan data mobil
public void tampilkanInfo() {
    System.out.println("Mobil: " + nama + " [" + merk + "], Tahun: " + tahun +
                       ", CC: " + cc + ", Harga: Rp" + harga);
}
```

a. Method ini digunakan untuk menampilkan informasi mobil ke layar

b. Menggabungkan semua atribut dalam satu output

6. Method main(opsional)

```java
    public static void main(String[] args) {
        // TODO Auto-generated method stub

    }

}
```

a. Tidak digunakan dalam class ini

b. Biasanya dipakai untuk testing

c. Dalam program kamu, main sudah ada di Driver, jadi ini boleh kosong atau dihapus

2. Kelas DriverMobil_2511538002:

1.

```java
package pekan1_2511538002;

import java.util.ArrayList;

public class DriverMobil_2511538002 {

```

a. Package pekan1_2511538002 digunakan untuk megelompokan class ke dalam satu folder/ project agar lebih trustruktur

b. Import java.util.arrayList: dugunakan untuk mengimpor class Arraylist yaitu struktur data yang dapat menyimpan banyak objek dan ukurannya bisa berubah secara dinamis.

c. Yang ketiha itu tentang : Class ini berfungsi sebagai **driver (pengendali utama)** untuk menjalankan program.

1. Deklarasi Atribut dan method tambah mobil

```java
// List untuk menampung objek mobil
private ArrayList<mobil_2511538002> daftarMobil = new ArrayList<>();

// Method Tambah Mobil
public void tambahMobil(mobil_2511538002 m) {
    daftarMobil.add(m);
    System.out.println("Berhasil menambahkan: " + m.getNama());
}
```

a. private → hanya bisa diakses di dalam class , ArrayList<Mobil> → menyimpan objek bertipe Mobil, daftarMobil → nama variabel list. Fungsi: Menyimpan semua data mobil yang diinput.

b. Method ini digunakan untuk menambahkan objek mobil ke dalam list
add(m) → menambahkan data ke ArrayList
getNama() → mengambil nama mobil

2. Method hapus mobil

```java
// Method Hapus Mobil
public void hapusMobil(int index) {
    if (index >= 0 && index < daftarMobil.size()) {
        mobil_2511538002 removed = daftarMobil.remove(index);
        System.out.println("Berhasil menghapus: " + removed.getNama());
    } else {
        System.out.println("Gagal menghapus: Indeks tidak ditemukan.");
    }
}
```

a. Menghapus mobil berdasarkan index

b. size() → jumlah data dalam list

c. remove(index) → menghapus data

d. Menggunakan if untuk mencegah error (index tidak valid)

3. Method tampilkan semua mobil

```
    // Method Tampilkan Semua
    public void tampilkanSemuaMobil() {
        System.out.println("\n--- DAFTAR MOBIL ---");
        for (int i = 0; i < daftarMobil.size(); i++) {
            System.out.print((i + 1) + ". ");
            daftarMobil.get(i).tampilkanInfo();
        }
    }
```

a. Menampilkan seluruh data mobil

b. Menggunakan looping (for)

c. get(i) → mengambil data berdasarkan index

d. tampilkanInfo() → method dari class Mobil

4. Method main program utama, mebuat obkek driver

```
public static void main(String[] args) {
    DriverMobil_2511538002 driver = new DriverMobil_2511538002();
```

a. Puvlic static merupakan titik awal program dijalankan

b. DriverMobil_2511538002 digunakan untuk mengaksese method dalam class.

5. Membuat objek mobil

```
    mobil_2511538002 mobil1 = new mobil_2511538002("Avanza", 2020, 1300, 150000000L, "Toyota");
    mobil_2511538002 mobil2 = new mobil_2511538002("Civic", 2022, 1500, 600000000L, "Honda");
```

a. • Membuat objek dari class Mobil, Parameter: Nama mobil, Tahun, Kapasitas mesin, Harga, Merek

6. Menabah mobil

```
driver.tambahMobil(mobil1);
driver.tambahMobil(mobil2);
```

7.

```
        driver.tampilkanSemuaMobil();

        mobil1.setHarga(155000000L);
        System.out.println("\n(Update) Harga " + mobil1.getNama() + " diubah.");

        driver.hapusMobil(1);

        driver.tampilkanSemuaMobil();
    }
}
```

a. Menampilkan data

b. Mengubah data(mutator) : Menggunakan method setter dan Mengubah harga mobil.

c. Menghapus data : menghapus mobil ke-2(index dimulai dari 0)

d. Menampilkan hasil akhir

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Abstract Data Type (ADT) dapat diimplementasikan dengan baik dalam bahasa Java melalui pembuatan class yang merepresentasikan objek Jam.
2. Program berhasil mengelola data waktu (jam, menit, detik) serta menyediakan operasi penting seperti validasi, konversi ke detik, perbandingan, dan manipulasi waktu.
3. Penerapan konsep enkapsulasi melalui atribut private serta penggunaan getter dan setter membantu menjaga keamanan dan struktur data.
4. Program driver menunjukkan bahwa ADT dapat digunakan secara nyata, baik melalui data langsung maupun input dari pengguna.
5. Masih terdapat beberapa kekurangan, seperti ketidakkonsistenan penamaan method dan kesalahan pada fungsi penjumlahan (plus), sehingga perlu perbaikan lebih lanjut.

DAFTAR PUSTAKA

Herbert Schildt. 2014. *Java: The Complete Reference*. New York: McGraw-Hill Education.

Y. Daniel Liang. 2013. *Introduction to Java Programming*. Boston: Pearson Education.

Oracle Corporation. 2023. *Java Documentation*. Diakses dari: https://docs.oracle.com

Data Structures and Algorithms in Java. 2014. Hoboken: John Wiley & Sons.